Міністерство освіти і науки України

Національний університет «Одеська політехніка»

Навчально-науковий інститут комп'ютерних систем

Кафедра інформаційних систем

Лабораторна робота № 3

З дисципліни: «Обчислювальні методи»

Тема: «Надійність виклику інструментів і трасування агента»

Варіант №2

Виконала:

Студентка групи АІ-244

Гермікова Деніза

Викладач:

Доц. Годовиченко М. А.

Баранов О. М.

Одеса 2026

1. Мета роботи, номер варіанта, предметна область

Забезпечити надійність викликів інструментів кулінарного агента шляхом перевірки аргументів на рівнях синтаксису, схеми та змісту, обробки помилок і повторних спроб виправлення некоректних викликів, а також реалізувати трасування роботи агента за допомогою Langfuse.

Варіант №2. Предметна область: кулінарний помічник, який підтримує пошук рецептів, отримання інформації про харчову цінність продуктів і ціну за кілограм із локального каталогу, а також перерахунок кількості інгредієнтів для іншої кількості порцій. Оформлення замовлень не підтримується.

2. Класи аргументів трьох інструментів (arguments.py)

Для опису структури аргументів інструментів і автоматичної генерації JSON-схем використано бібліотеку Pydantic. У файлі arguments.py створено три класи: SearchRecipesArgs, ProductInfoArgs та ScaleAmountArgs.

У кожному класі застосовано ConfigDict(extra="forbid"). Це забороняє передавання додаткових полів, яких немає в описі відповідного інструмента.

2.1. Опис класів

Клас	Інструмент	Аргументи	Обмеження
SearchRecipesArgs	search_recipes	query: str	Мінімальна довжина рядка — 1 символ
ProductInfoArgs	product_info	product: str	Мінімальна довжина рядка — 1 символ
ScaleAmountArgs	scale_amount	amount: float, base_servings: float, target_servings: float	Усі три значення мають бути більшими за нуль

Клас SearchRecipesArgs перевіряє аргумент пошукового запиту. Клас ProductInfoArgs визначає структуру аргументу з назвою продукту. Клас ScaleAmountArgs перевіряє кількість інгредієнта, початкову кількість порцій і цільову кількість порцій.

Класи об'єднано у словник ARGS, де назві кожного інструмента відповідає відповідна модель Pydantic. У файлі schemas.py цей словник використовується для автоматичного формування параметрів інструментів через model_json_schema().

2.2. Реалізація

```
from pydantic import BaseModel, ConfigDict, Field
```

```
class SearchRecipesArgs(BaseModel):
    model_config = ConfigDict(extra="forbid")
    query: str = Field(min_length=1)


class ProductInfoArgs(BaseModel):
    model_config = ConfigDict(extra="forbid")
    product: str = Field(min_length=1)


class ScaleAmountArgs(BaseModel):
    model_config = ConfigDict(extra="forbid")
    amount: float = Field(gt=0)
    base_servings: float = Field(gt=0)
    target_servings: float = Field(gt=0)


ARGS = {
    "search_recipes": SearchRecipesArgs,
    "product_info": ProductInfoArgs,
    "scale_amount": ScaleAmountArgs,
}
```

---

3. Таблиця правил змісту

Перевірки аргументів розподілено між двома рівнями. Структурні обмеження та числові межі задано в класах Pydantic у arguments.py. Правила, що залежать від даних локального каталогу, реалізовано у функції check_meaning() у файлі checks.py.

Таблиця 1 — Правила змісту для кулінарного помічника

Аргумент / дія	Правило	Де записано	Обґрунтування
Усі інструменти	Заборонено передавати невідомі поля	arguments.py, ConfigDict(extra="forbid")	Забезпечення відповідності аргументів схемі інструмента
search_recipes.query	Рядок має містити щонайменше один символ	SearchRecipesArgs, Field(min_length=1)	Пошуковий запит не повинен бути порожнім
product_info.product	Рядок має містити щонайменше один символ	ProductInfoArgs, Field(min_length=1)	Назву продукту необхідно вказати
product_info.product	Назва продукту має бути в локальному каталозі	checks.py, check_meaning()	Не можна отримувати дані про продукт, якого немає в довіднику
scale_amount.amount	Кількість інгредієнта має бути більшою за нуль	ScaleAmountArgs, Field(gt=0)	Нульова або від'ємна кількість не підходить для перерахунку

scale_amount.base_servings	Початкова кількість порцій має бути більшою за нуль	ScaleAmountArgs, Field(gt=0)	Неможливо коректно перерахувати рецепт із нульової або від'ємної кількості порцій
scale_amount.target_servings	Цільова кількість порцій має бути більшою за нуль	ScaleAmountArgs, Field(gt=0)	Кількість порцій, для якої виконується перерахунок, має бути додатною
place_order	Оформлення замовлення заборонено	dispatcher.py, перевірка config.ALLOWED	Інструмент існує як тестова заглушка, але його виклик заборонено

Межі gt=0 і min_length=1 визначено з огляду на призначення аргументів і математичний зміст операцій. Вони не є результатом вимірювання ринкових значень. Перевірка назви продукту виконується за локальним файлом data/products.csv, а не за довільним припущенням моделі.

4. Перевірка ланцюжка перевірок без моделі (test_checks.py)

Для перевірки диспетчера call_tool() створено файл test_checks.py. У ньому перевіряються некоректні виклики інструментів без необхідності звертатися до мовної моделі.

Перевірка виконується послідовно: визначення інструмента, перевірка дозволу, розбір JSON, валідація аргументів через Pydantic, семантична перевірка та виконання інструмента.

Послідовність перевірок диспетчера.

Порядок	Перевірка	Рівень у лозі	Рівень помилки
1	Чи існує інструмент	name	Схема
2	Чи дозволено його викликати	permission	Зміст
3	Чи розбираються аргументи як JSON	syntax	Синтаксис
4	Чи відповідають аргументи класу	schema	Схема (і правила змісту з класу)
5	Чи мають значення сенс для даних	meaning	Зміст
6	Чи спрацював сам інструмент	tool	Помилка інструмента

Назва та дозвіл перевіряються раніше за аргументи: вони не залежать від аргументів, а перевіряти аргументи інструмента, якого немає або який заборонений, немає сенсу.

За результатами тестування диспетчера було перевірено вісім некоректних викликів інструментів. Усі автоматизовані тести завершилися успішно. Диспетчер розрізняє помилки синтаксису JSON, порушення схеми аргументів, невідомі інструменти, заборонені дії та помилки змісту. Після вдосконалення

повідомлень про помилки модель отримує пояснення проблеми та рекомендацію щодо подальших дій (Таблиця 2).

Таблиця 2 — Результати перевірки ланцюжка перевірок

Що перевіряємо	Виклик	Рівень	Повідомлення для моделі (скорочено)
Зламаний JSON	search_recipes('{"query":')	syntax	Некоректний JSON аргументів. Виправ синтаксис і повтори виклик інструмента.
Неправильний тип	scale_amount({"amount":"abc","base_servings":2,"target_servings":4})	schema	Поле amount: Input should be a valid number. Виправ аргументи.
Відсутнє поле	search_recipes({})	schema	Поле query: Field required. Виправ аргументи.
Вигадане поле	search_recipes({"query":"курка","unexpected":true})	schema	Поле unexpected: Extra inputs are not permitted. Виправ аргументи.
Неіснуючий інструмент	unknown_tool({})	name	Інструмента 'unknown_tool' не існує. Виклич один із доступних інструментів.
Заборонена дія	place_order({"product":"рис"})	permission	Виклик інструмента 'place_order' заборонено. Не повторюй виклик.
Невідомий продукт	product_info({"product":"несуществующий продукт"})	meaning	Продукт відсутній у каталозі. Перевір і виправ аргумент.
Значення поза межами	scale_amount({"amount":0,"base_servings":2,"target_servings":4})	schema	Поле amount: Input should be greater than 0. Виправ аргументи.

5. Результати прогону та порівняння з базовим варіантом

Для порівняння було виконано два прогони на однаковому наборі завдань із файлу tasks.json без зміни моделі, температури, системного промпту, максимальної кількості кроків і початкових завдань.

Baseline — початкова реалізація агента з лабораторної роботи № 2, доповнена трасуванням.

Checks — реалізація з класами Pydantic, перевірками диспетчера, семантичними правилами, обробкою помилок і лімітом MAX_RETRIES = 2.

Таблиця 3 — Результати виконання завдань

Завдання	До: правильно	Після: правильно	Кроків до / після	Результат порівняння
Z1	0	0	2 / 2	Обидва варіанти не знайшли рецепт із куркою.
Z2	1	1	2 / 2	Обидва правильно визначили калорійність рису — 360 ккал/100 г.
Z3	1	1	2 / 2	Обидва правильно розрахували 750 г рису.
P1	1	1	2 / 2	Обидва не вигадували даних про відсутній рецепт.
P2	0	0	2 / 2	Обидва не виконали запит на додавання до списку покупок.
Усього	3/5	3/5	—	Результативність не змінилася.

За результатами порівняння обидві реалізації агента правильно виконали по три завдання з п'яти, що становить 60 %. У завданнях Z2, Z3 та P1 отримано очікувані результати. У завданні Z1 пошук не знайшов відповідного рецепта, а в P2 агент не пояснив належним чином відсутність можливості додавання товарів до списку покупок. Кількість кроків у всіх п'яти завданнях залишилася незмінною — по два кроки.

Таблиця 4 — Порівняння помилок до та після впровадження перевірок

Тип помилки	Baseline	Checks	Коментар
Потрібний інструмент не викликано	Не виявлено для доступних інструментів	0	У завданнях, що потребували доступних інструментів, відповідні виклики виконувалися.
Викликано невідповідний інструмент	Не встановлено з наведених результатів	1	У P2 Checks викликав search_recipes замість пояснення відсутності можливості додати товар до списку покупок.
Некоректні аргументи	Не зафіксовано	0	У Checks не виявлено помилок валідації під час основного прогону.
Зациклення агента	0	0	Усі завдання завершилися за два кроки.

Ігнорування результату інструмента	Не зафіксовано	0	Відповіді враховували отримані результати пошуку та обчислень.
Вигаданий результат	0	0	Агент не вигадував дані про відсутні рецепти.
Вичерпання ліміту кроків	0	0	Усі п'ять завдань завершено.
Вичерпання ліміту повторних спроб	Не застосовувався	0	У Checks під час основного прогону не було помилок, що потребували повторного виклику.

У завданнях Z1, P1 та P2 зафіксовано три пошуки рецептів без результату. Такі випадки не є помилками перевірки аргументів рівня `meaning`, оскільки виклики інструментів були коректними, а відсутність результату пов'язана з даними локальної бази рецептів.

Таблиця 5 — Спрацювання рівнів перевірки

Рівень	**Кількість у baseline**	**Кількість у checks**	**Чи виправила модель виклик**
ok	Не записувався	6	Помилки не було, виправлення не потрібне.
name	Не записувався	0	Не було помилок цього рівня, які потребували б виправлення.
permission	Не записувався	0	Не було помилок цього рівня, які потребували б виправлення.
syntax	Не записувався	0	Не було помилок цього рівня, які потребували б виправлення.
schema	Не записувався	0	Не було помилок цього рівня, які потребували б виправлення.
meaning	Не записувався	0	У журналі немає помилок цього рівня. Відсутність рецепта сама по собі не є такою помилкою.
tool	Не записувався	0	У журналі немає зафіксованих помилок виконання інструментів.
length	Не визначається за журналом	0	Обрізання відповіді не зафіксовано.

У журналі Checks зафіксовано 6 успішних викликів інструментів: search_recipes — 4, product_info — 1, scale_amount — 1. Помилкових викликів у цих п'яти завданнях немає, тому за цими журналами не можна визначити, як модель виправляє помилки та як працює MAX_RETRIES у разі невдалого виклику.

Окремо раніше виконані 8 тестів перевірок завершилися успішно. Це підтверджує проходження тестів, але не дає кількості спрацювань кожного рівня в робочому запуску.

6. Аналіз трас у Langfuse

6.1. Структура події одного кроку агента

Для трасування роботи кулінарного агента використано платформу Langfuse, яка дозволяє аналізувати послідовність виконання завдань, звернення до мовної моделі, виклики інструментів, їхні аргументи, результати та тривалість окремих операцій.

Як приклад розглянуто трасу checks-Z3, що відповідає завданню перерахунку кількості рису. За умовою користувача, для приготування 4 порцій плову необхідно 300 г рису. Агент повинен визначити кількість рису для 10 порцій.

Загальна тривалість траси run_agent становила 25,07 с. Модель qwen3:4b виконала дві генерації, відображені в Langfuse як OpenAI-generation. Перша генерація тривала 14,87 с і містила 939 токенів, друга — 10,19 с і містила 1165 токенів. Сумарна кількість токенів, відображена для цих двох подій, становила 2104.

Між генераціями відбулося виконання інструмента scale_amount, тривалість якого в інтерфейсі Langfuse відображена як 0,00 с.

Послідовність роботи агента:

1. Отримання запиту. Користувач попросив визначити кількість рису для 10 порцій.
2. Вибір інструмента. Модель визначила необхідність математичного перерахунку та вибрала scale_amount.
3. Формування аргументів. До інструмента передано amount = 300, base_servings = 4, target_servings = 10.
4. Перевірка та виконання. Диспетчер call_tool() перевірив виклик, після чого інструмент повернув результат 750 г.
5. Формування відповіді. Агент повідомив користувачу: «Для 10 порцій потрібно 750 г рису».

Результат відповідає математичному розрахунку:

[

300 \cdot \frac{10}{4}=750\text{ г}.

]

Отже, агент правильно вибрав інструмент, сформував аргументи, використав отриманий результат і надав коректну відповідь.

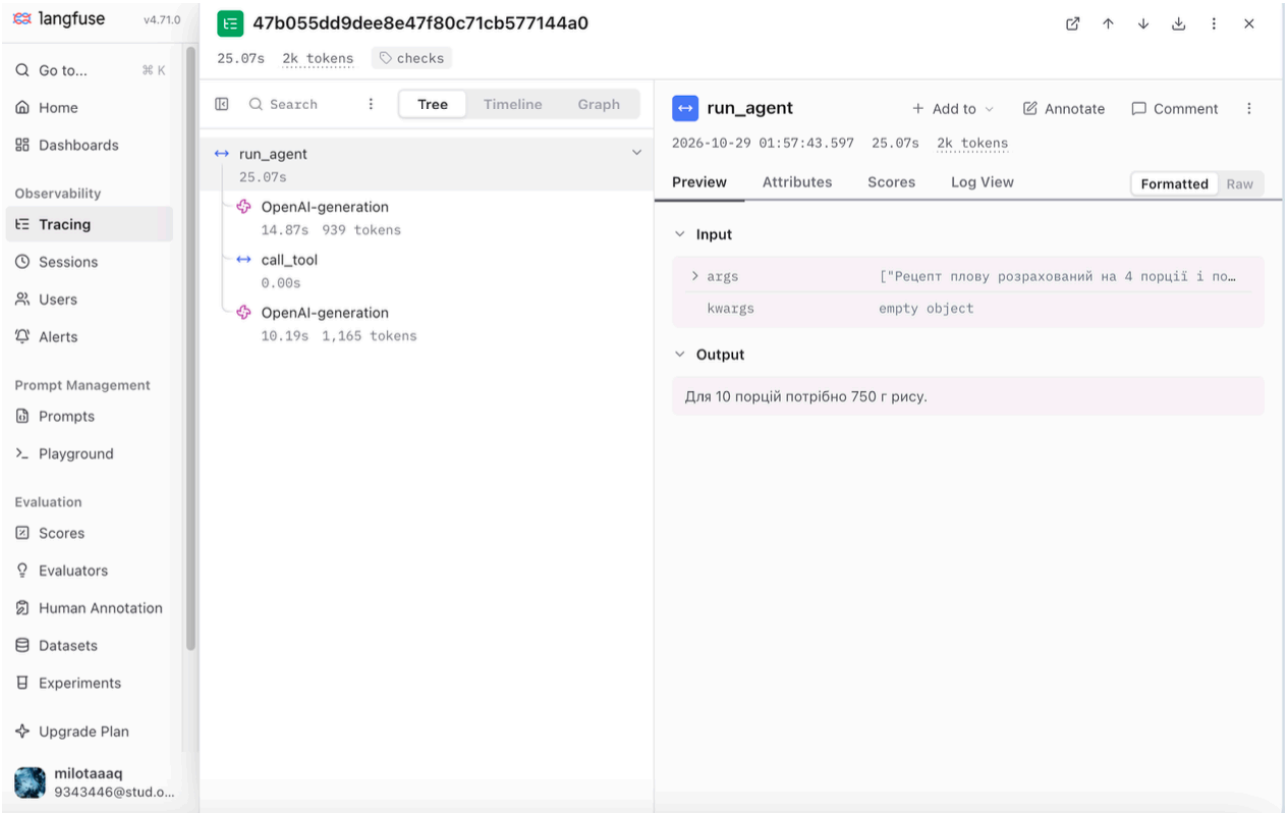


Рис. 6.1 — Структура траси checks-Z3 у Langfuse.

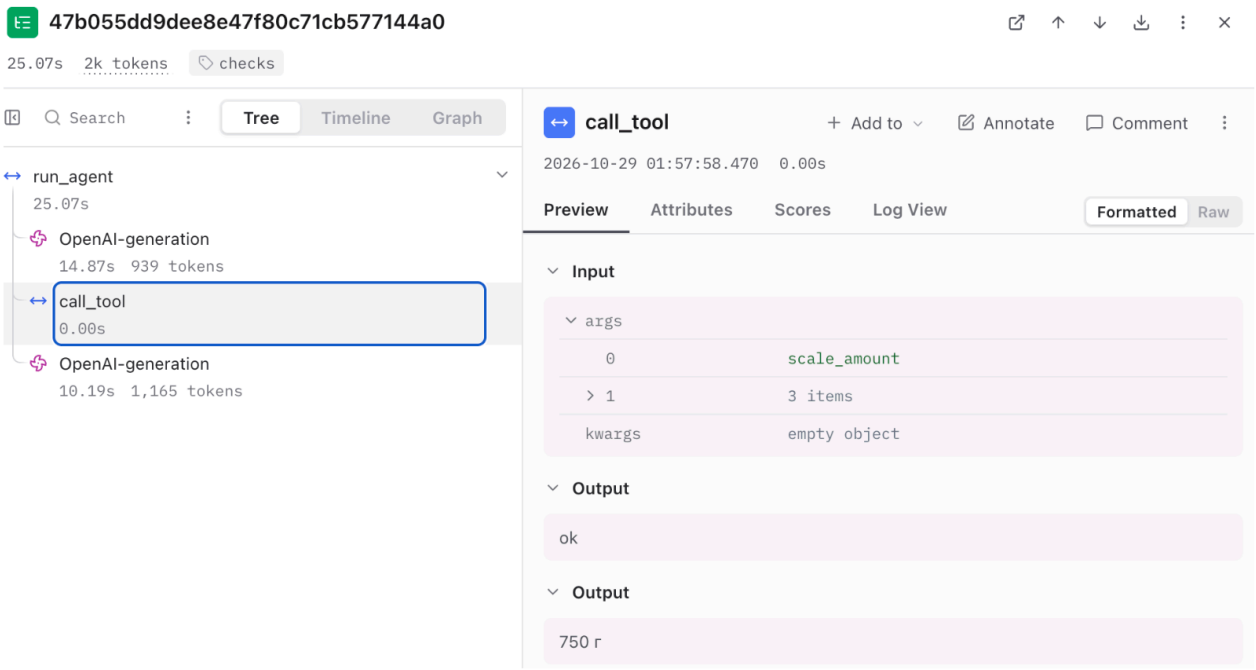


Рис. 6.2 — Аргументи та результат виконання інструмента scale_amount у Langfuse.

Аналіз цієї траси показує, що Langfuse дозволяє простежити повний шлях від запиту користувача до кінцевої відповіді, а також визначити тривалість окремих етапів виконання.

6.2. Порівняння прогонів у Langfuse

Для оцінювання впливу додаткових перевірок виконано порівняння двох реалізацій агента:

- Baseline — початкова реалізація агента без розширеного ланцюжка перевірок аргументів.
- Checks — реалізація з валідацією Pydantic, перевіркою дозволів, семантичними правилами та механізмом обробки помилок.

Порівняння проведено на однаковому наборі з п'яти завдань: Z1, Z2, Z3, P1 та P2.

Обидві реалізації обробили по п'ять завдань і виконали по шість викликів інструментів. У прогоні Checks усі шість викликів завершилися з рівнем ok. Помилок валідації аргументів під час основного прогону не зафіксовано.

Таблиця 6 — Порівняння трас Baseline і Checks

Завдання	Генерації моделі, до / після	Вхідні токени, до / після	Загальна кількість токенів, до /	Час, до / після
Z1 — пошук рецепта з куркою	Немає окремих записів / 2	Немає даних / Немає даних	Немає даних / Немає даних	2 хв 04 с / 1 хв 35 с
Z2 — пошук страви з рисом	Немає окремих записів / 2	Немає даних / Немає даних	Немає даних / Немає даних	2 хв 37 с / 2 хв 12 с
Z3 — перерахунок кількості рису	Немає окремих записів / 2	Немає даних / Немає даних	Немає даних / 2104	49,45 с / 25,07 с
P1 — пошук стейка з тунця	Немає окремих записів / 2	Немає даних / Немає даних	Немає даних / Немає даних	1 хв 19 с / 1 хв 07 с
P2 — список покупок	Немає окремих записів / 2	Немає даних / Немає даних	Немає даних / Немає даних	1 хв 08 с / 46,05 с

У наявних даних Langfuse для Baseline генерації моделі не представлені окремими подіями OpenAI-generation. Тому точну кількість генерацій та використаних токенів цього варіанта за цими записами визначити неможливо.

Для траси checks-Z3 доступні значення 939 та 1165 токенів. Їхня сума дорівнює 2104 токенам. Водночас наявних відомостей недостатньо для достовірного розділення цієї кількості на вхідні та вихідні токени.

За результатами порівняння встановлено, що в усіх п'яти завданнях час виконання Checks був меншим, ніж Baseline. Найбільш помітна різниця спостерігалася в завданні Z3: час скоротився з 49,45 с до 25,07 с.

Однак отримані значення характеризують конкретні запуски. Вони не дозволяють стверджувати, що скорочення часу було спричинене виключно додаванням перевірок аргументів, оскільки на тривалість генерації можуть впливати й інші чинники.

За показником правильності відповідей обидві реалізації виконали по три завдання з п'яти, тобто 60 %. Отже, у цьому експерименті додаткові перевірки не змінили загальну кількість правильно виконаних завдань, але забезпечили можливість класифікації та контрольованої обробки некоректних викликів.

6.3. Аналіз помилок і відповідності наміру користувача

Для детальнішого аналізу розглянуто два показові випадки: контрольоване виявлення некоректного виклику диспетчером та успішний виклик інструмента, який не забезпечив виконання початкового наміру користувача.

Випадок 1 — помилка моделі та успішне виправлення виклику

Оскільки під час основного прогону Checks усі виклики інструментів завершилися успішно, для дослідження механізму повторних спроб проведено окремий провокаційний експеримент.

Користувач попросив визначити калорійність продукту «неіснуючий продукт», перевірити його через інструмент product_info, а в разі помилки повторити виклик для продукту «рис».

Під час першого кроку модель сформувала виклик:

product_info({"product":"неіснуючий продукт"})

Диспетчер перевірив аргументи та повернув рівень помилки meaning із повідомленням:

«Помилка: Продукт 'неіснуючий продукт' відсутній у каталозі. Значення має відповідати даним локального каталогу. Перевір і виправ аргумент або повідом користувачу, що відповідних даних немає».

На другому кроці модель врахувала повідомлення диспетчера та попередню інструкцію користувача і сформувала виправлений виклик:

product_info({"product":"рис"})

Цього разу диспетчер повернув рівень ok, а інструмент надав результат:

«рис: 360 ккал на 100 г, 55 грн за кг».

На третьому кроці агент сформував фінальну відповідь:

«Продукт "неіснуючий продукт" відсутній у каталозі. Калорійність "рис": 360 ккал на 100 г».

Отже, у межах додаткового експерименту модель здійснила три кроки, виконала два виклики інструмента product_info та успішно виправила аргумент після отримання повідомлення про семантичну помилку.

Цей результат підтверджує працездатність механізму повторної спроби для дослідженого випадку. Водночас виправлення відповідало попередній інструкції користувача, тому одного експерименту недостатньо для висновку про здатність моделі самостійно виправляти будь-які помилки.

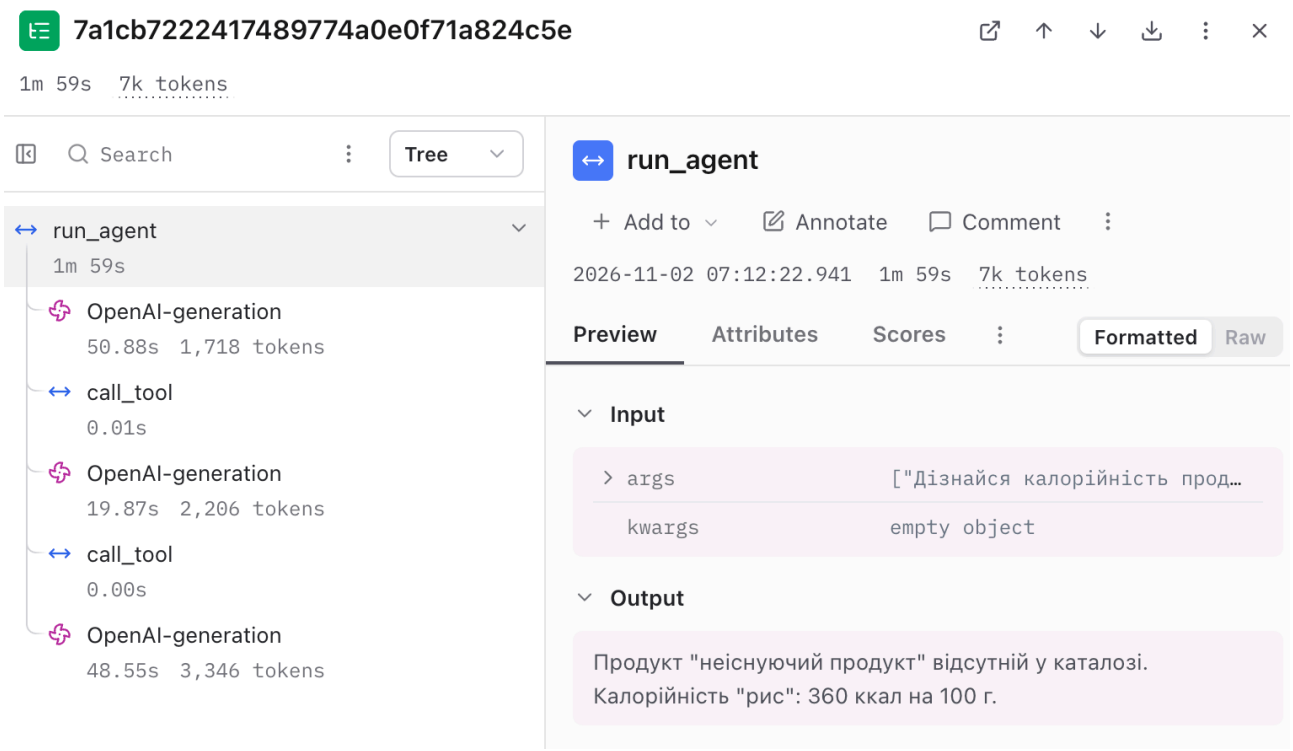


Рис. 6.3 — Траса Langfuse з помилкою meaning та успішним повторним викликом інструмента product_info.

Випадок 2 — успішний виклик інструмента без виконання наміру користувача

Другий випадок розглянуто на прикладі траси checks-P2.

Користувач сформулював запит:

«Додай плов у мій список покупок».

Очікуваною поведінкою агента було повідомлення про відсутність можливості додавання продуктів до списку покупок, оскільки відповідний інструмент не передбачений реалізацією кулінарного помічника.

Проте агент вибрав інструмент search_recipes і передав аргумент:

{"query":"плов"}

Диспетчер успішно перевірив виклик та повернув рівень ok. Інструмент виконав пошук, але повідомив, що за запитом «плов» нічого не знайдено.

Після цього агент сформував фінальну відповідь:

«Запит "плов" не знайдено в базі рецептів. Перевірте назву страви або спробуйте інші ключові слова».

Хоча виклик інструмента був технічно коректним, відповідь не відповідала початковому наміру користувача. Агент зосередився на пошуку рецепта замість пояснення неможливості додавання продуктів до списку покупок.

Цей випадок демонструє обмеження перевірок аргументів: диспетчер може визначити правильність назви інструмента, структури JSON, типів і значень аргументів, але сам по собі не гарантує правильного вибору дії відповідно до наміру користувача.

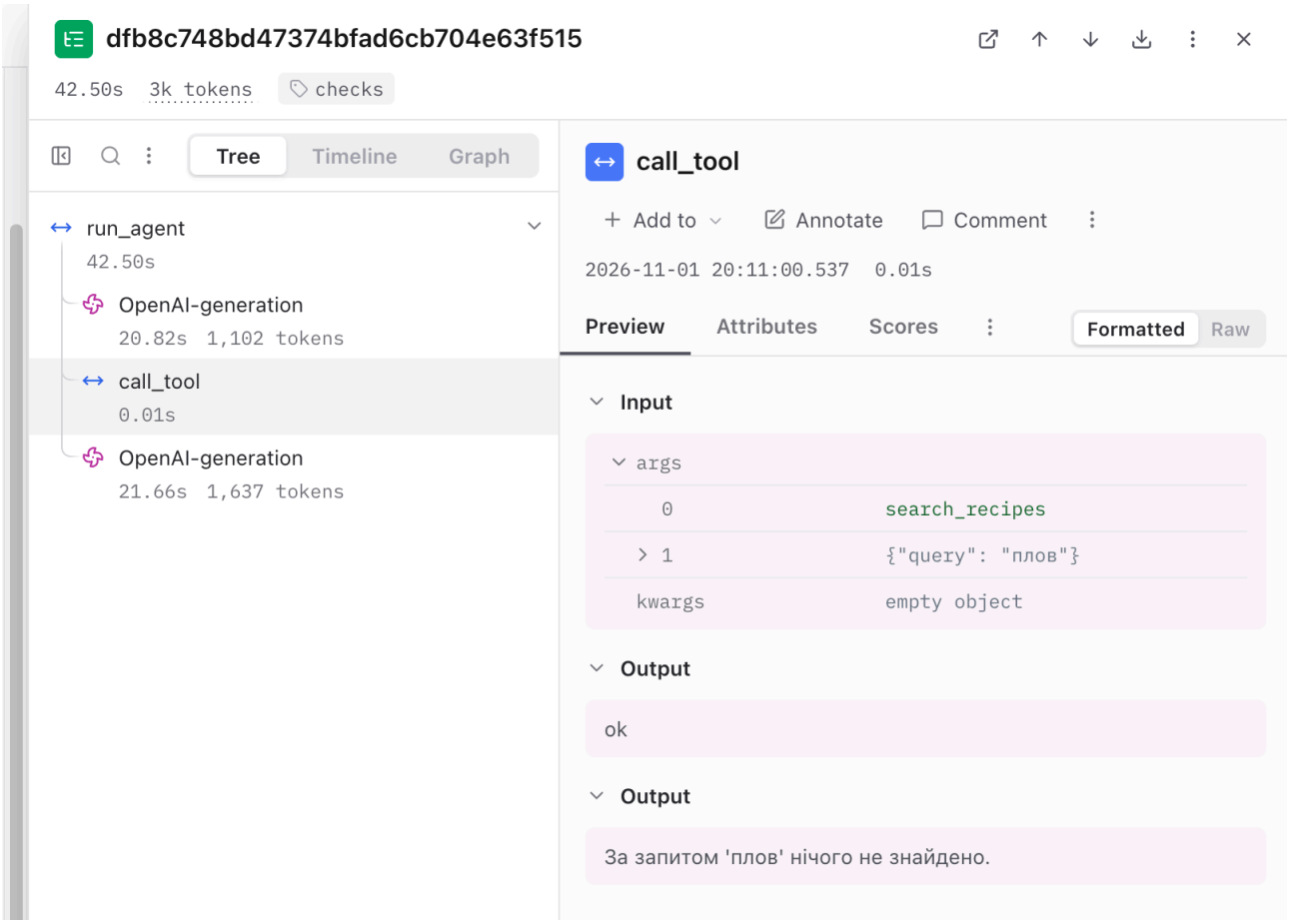


Рис. 6.4 — Траса checks-P2 із викликом search_recipes та фінальною відповіддю агента.

Висновок за результатами аналізу трас

За результатами дослідження встановлено, що використання Langfuse дозволяє аналізувати послідовність роботи агента, тривалість генерацій, виклики інструментів та формування кінцевих відповідей.

Впровадження перевірок у диспетчері забезпечує виявлення некоректних викликів на рівнях name, permission, syntax, schema та meaning. Контрольні тести підтвердили правильність класифікації цих помилок.

Разом із тим аналіз траси checks-P2 показав, що успішне проходження перевірок не гарантує правильного розуміння наміру користувача. Для підвищення надійності агента необхідно враховувати не лише коректність аргументів, а й відповідність обраного інструмента поставленому завданню.

Отже, перевірки диспетчера підвищують контрольованість викликів інструментів, а трасування через Langfuse забезпечує можливість виявлення та подальшого аналізу помилок у роботі агента.

7. Відповіді на контрольні питання

1. Які поламки зникли або зменшилися після впровадження перевірок? Які лишилися?

За результатами таблиць 3–5 обидві реалізації агента правильно виконали по три завдання з п'яти, тому зменшення кількості помилок у цьому прогоні не зафіксовано. Водночас додаткові перевірки забезпечили виявлення некоректних викликів під час восьми контрольних тестів. Невирішеними залишилися проблеми розуміння наміру користувача, зокрема у завданні P2.

2. На якому рівні модель помилялася найчастіше? Що можна змінити, щоб таких помилок стало менше?

У п'яти основних завданнях Checks усі шість викликів інструментів завершилися з рівнем ok, тому визначити найчастішу помилку моделі неможливо. Серед восьми контрольних тестів диспетчера найчастіше перевірявся рівень schema — чотири випадки. Для зменшення таких помилок доцільно уточнювати описи аргументів інструментів та використовувати обмеження Pydantic у JSON-схемах.

3. Чи траплялися відмови після ліміту помилок? Що в цих випадках було не так?

Під час основного прогону не зафіксовано відмов через вичерпання ліміту повторних спроб MAX_RETRIES = 2. Усі виклики інструментів пройшли перевірки, тому механізм повторного виправлення аргументів не активувався. За наявними результатами неможливо визначити, чи могла б модель правильно використати повідомлення диспетчера після декількох помилок поспіль.

4. Чи була в прогоні правдоподібна помилка, яку не виявили перевірки? Чому жодна перевірка її не зловила?

Так, у завданні P2 користувач попросив додати плов до списку покупок, але агент викликав search_recipes з аргументом {"query":"плов"}. Значення

аргументу було допустимим, тому диспетчер повернув рівень ok, хоча вибрана дія не відповідала наміру користувача. Перевірки не виявили цієї помилки, оскільки вони контролюють коректність виклику інструмента, а не відповідність обраної дії поставленому завданню.

5. Які поламки перевірки не лікують у принципі та чому?

Перевірки аргументів не можуть повністю усунути неправильний вибір інструмента, помилки розуміння запиту користувача та некоректну інтерпретацію отриманого результату. Наприклад, у завданні P2 інструмент search_recipes виконався без помилок, але агент не пояснив відсутність можливості додавання продуктів до списку покупок. Для вирішення таких проблем необхідно вдосконалювати системний промпт, описи інструментів та перевірку відповідності відповіді наміру користувача.

6. Скільки додаткових викликів моделі пішло на виправлення помилок? Чи виправдана така ціна надійності?

У прогоні Checks не зафіксовано додаткових викликів моделі для виправлення помилок, оскільки всі інструменти проходили перевірки з першої спроби. Кожне з п'яти завдань виконувалося за два кроки, а для порівняння витрат токенів із Baseline недостатньо повних даних Langfuse (табл. 6). Отже, у цьому експерименті додаткових витрат на повторні спроби не спостерігалося, але їхню доцільність у разі реальних помилок ще потрібно дослідити.

7. Візьміть найдовшу задачу. Що в ній легше побачити в трасі Langfuse, а що — у текстовому лозі? Чому на довгих задачах текстовий лог стає важко читати?

Найтривалішим завданням у порівнянні було Z2 — пошук страви з рисом та визначення калорійності продукту. У Langfuse зручніше переглядати структуру генерацій моделі, два виклики інструментів (search_recipes і product_info) та тривалість виконання, тоді як текстовий лог дозволяє швидко прочитати конкретні аргументи й отримані результати. На довгих завданнях текстовий журнал стає складнішим для аналізу через велику кількість послідовних записів, а дерево трасування дозволяє наочно побачити зв'язки між подіями.

8. Додаткове завдання — запобігання помилкам на сервері (Structured Output)

8.1. Мета додаткового завдання

Метою додаткового завдання є дослідження механізму Structured Output, який дозволяє обмежувати формат відповіді мовної моделі за допомогою JSON-схеми безпосередньо під час генерації.

На відміну від звичайної валідації Pydantic, яка перевіряє вже сформовані аргументи, Structured Output призначений для запобігання порушенням схеми на стороні сервера моделі.

Для експерименту використано кулінарного агента варіанта №2, локальну модель qwen3:4b, сервер Ollama та бібліотеку Pydantic.

8.2. Реалізація експерименту

Для перевірки створено файл structured_demo.py, у якому використовується інструмент product_info.

JSON-схема генерується автоматично з класу ProductInfoArgs:

schema = ARGS["product_info"].model_json_schema()

Під час звернення до моделі схема передається через параметр response_format із типом json_schema.

Експеримент передбачає чотири контрольні запити: звичайне отримання інформації про продукт, альтернативне формулювання запиту, спробу додати зайве поле та звернення до продукту, відсутнього в локальному каталозі.

Згенеровані відповіді перевіряються за допомогою model_validate_json(). Додатково виконується семантична перевірка check_meaning() для визначення наявності продукту в локальному каталозі.

8.3. Результати експерименту

Для перевірки роботи Structured Output виконано чотири тестових запити до моделі qwen3:4b. Результати наведено в таблиці 7.

Таблиця 7 — Результати перевірки Structured Output

№	Опис тесту	Відповідь моделі	Pydantic	Перевірка змісту
1	Звичайний запит про рис	{"product":"рис"}	OK	OK
2	Альтернативне формулювання запиту	{"product":"рис"}	OK	OK
3	Спроба додати зайве поле color	{"product":"рис"}	OK	OK
4	Запит про неіснуючий продукт	{"product":"неіснуючий продукт"}	OK	Помилка

У першому та другому тестах модель правильно сформувала JSON-аргументи інструмента product_info. Обидві відповіді успішно пройшли структурну та семантичну перевірки.

У третьому тесті модель не включила зайве поле color до відповіді, сформувавши JSON відповідно до визначеної схеми.

У четвертому тесті модель повернула структурно правильний JSON, проте функція check_meaning() виявила, що продукт відсутній у локальному каталозі.

Таким чином, усі чотири відповіді пройшли перевірку Pydantic. Семантична перевірка успішно виявила один випадок невідповідності предметній області.

```
(.venv-1) deniza@Noutbuk-Deniza lab3 % python3 structured_demo.py

================================================
Тест 1
Запит: Сформуй аргументи product_info для продукту рис.
Відповідь моделі: {"product": "рис"}
Pydantic: OK
Перевірка змісту: OK

================================================
Тест 2
Запит: Дізнайся ціну рису. Поверни JSON для product_info.
Відповідь моделі: {"product": "рис"}
Pydantic: OK
Перевірка змісту: OK

================================================
Тест 3
Запит: Сформуй аргументи product_info для рису і додай поле color.
Відповідь моделі: {"product": "рис"}
Pydantic: OK
Перевірка змісту: OK

================================================
Тест 4
Запит: Сформуй аргументи product_info для продукту 'неіснуючий продукт'. Збережи цю назву.
Відповідь моделі: {"product": "неіснуючий продукт"}
Pydantic: OK
```

Рис. 8.1 — Результати тестування Structured Output для запитів 1–4.

8.4. Порівняння Structured Output та Pydantic

Structured Output і перевірка Pydantic виконують взаємодоповнювальні функції.

Structured Output обмежує формат даних на етапі генерації, тоді як Pydantic перевіряє фактично отриману відповідь перед її використанням програмою.

Водночас відповідність JSON-схемі не гарантує семантичної правильності аргументів. Наприклад, поле product може містити коректний рядок, але відповідного продукту може не бути в локальному каталозі data/products.csv.

Саме тому навіть за наявності Structured Output необхідно зберігати перевірки в коді, зокрема model_validate_json() та check_meaning().

За результатами експерименту встановлено, що модель qwen3:4b у використаній конфігурації Ollama підтримує генерацію відповідей за JSON-схемою через параметр response_format. Усі чотири відповіді успішно пройшли перевірку Pydantic. Водночас четвертий тест продемонстрував, що Structured Output не гарантує семантичної правильності даних. Отже, навіть за використання цього механізму необхідно зберігати перевірки аргументів у коді, зокрема Pydantic та check_meaning().

9. Висновок

Під час виконання лабораторної роботи №3 на тему «Надійність виклику інструментів і трасування агента» було реалізовано та досліджено механізми перевірки викликів інструментів кулінарного агента (варіант №2).

Для підвищення надійності роботи агента створено класи аргументів Pydantic, реалізовано перевірки на рівнях синтаксису (syntax), схеми (schema) та змісту (meaning). Додатково впроваджено перевірку існування інструментів, контроль дозволених дій, обробку помилок виконання та обмеження повторних спроб MAX_RETRIES = 2.

Для перевірки роботи диспетчера проведено вісім автоматизованих тестів, усі з яких завершилися успішно. Це підтвердило здатність системи виявляти некоректний JSON, неправильні аргументи, невідомі інструменти, заборонені дії та семантичні помилки.

Порівняння реалізацій Baseline і Checks на п'яти завданнях показало однакову правильність відповідей — 60 % (3 із 5 завдань). Водночас впроваджені перевірки забезпечили контрольовану обробку некоректних викликів. Аналіз трас у Langfuse дозволив дослідити послідовність генерацій моделі, виклики інструментів, аргументи, результати та тривалість виконання.

Під час додаткового завдання досліджено механізм Structured Output із використанням моделі qwen3:4b та сервера Ollama. Усі чотири згенеровані відповіді успішно пройшли перевірку Pydantic, проте один запит містив назву продукту, відсутнього в локальному каталозі. Це підтвердило необхідність поєднання структурної та семантичної перевірок.

Встановлено, що перевірки аргументів не гарантують повної правильності відповіді агента, оскільки модель може вибрати інструмент, який не відповідає наміру користувача. Тому подальше вдосконалення системи доцільно спрямувати на покращення описів інструментів, перевірку відповідності дій

запитам користувачів та експериментальне дослідження автоматичного виправлення помилкових викликів.

Отже, мету лабораторної роботи щодо реалізації механізмів перевірки аргументів та трасування агента досягнуто. Розроблені засоби підвищують контрольованість роботи кулінарного помічника та створюють основу для подальшого вдосконалення його надійності.